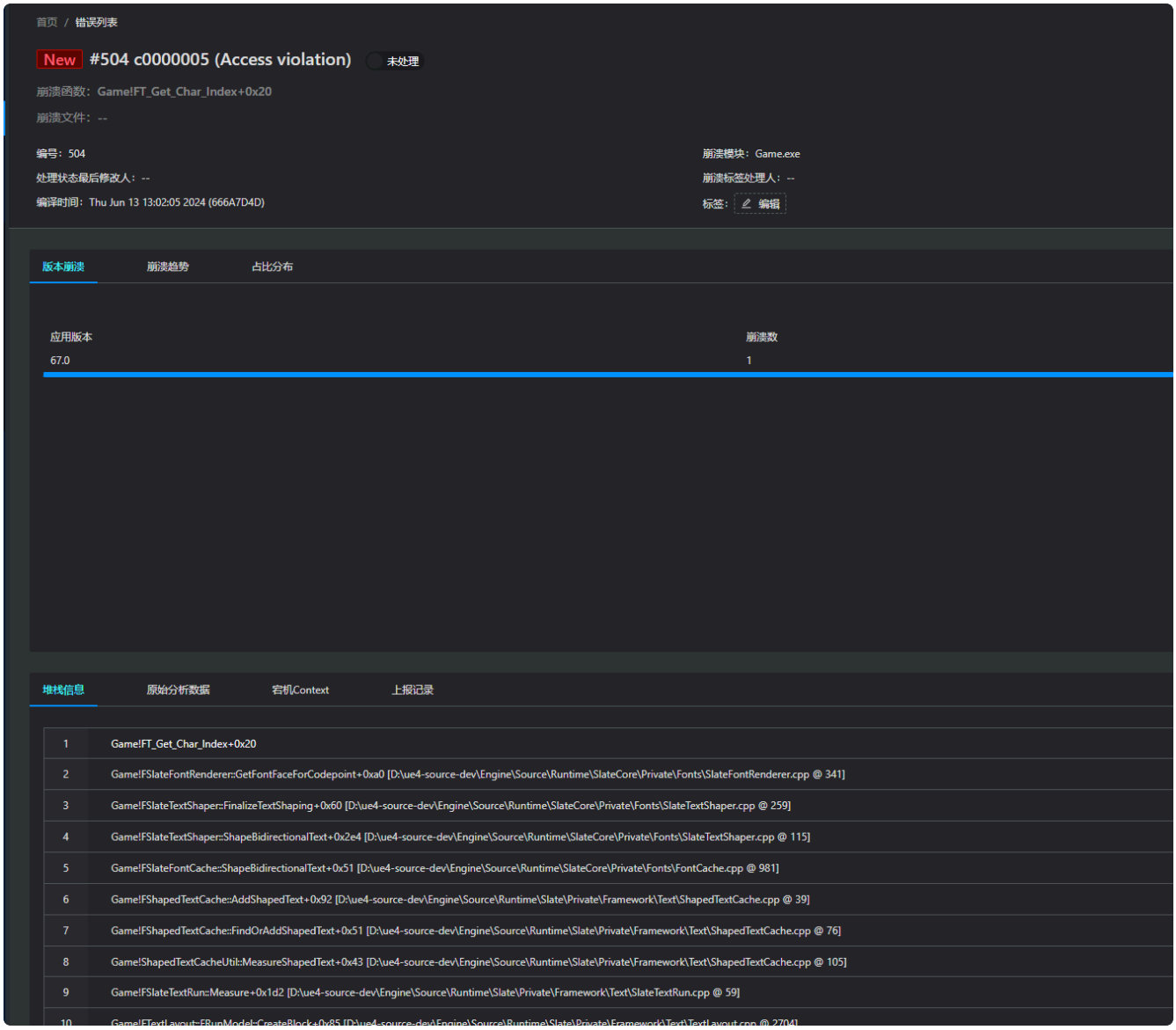
崩溃 / 卡死 / 死循环 排查方法

一、崩溃

1.1 CrashEye

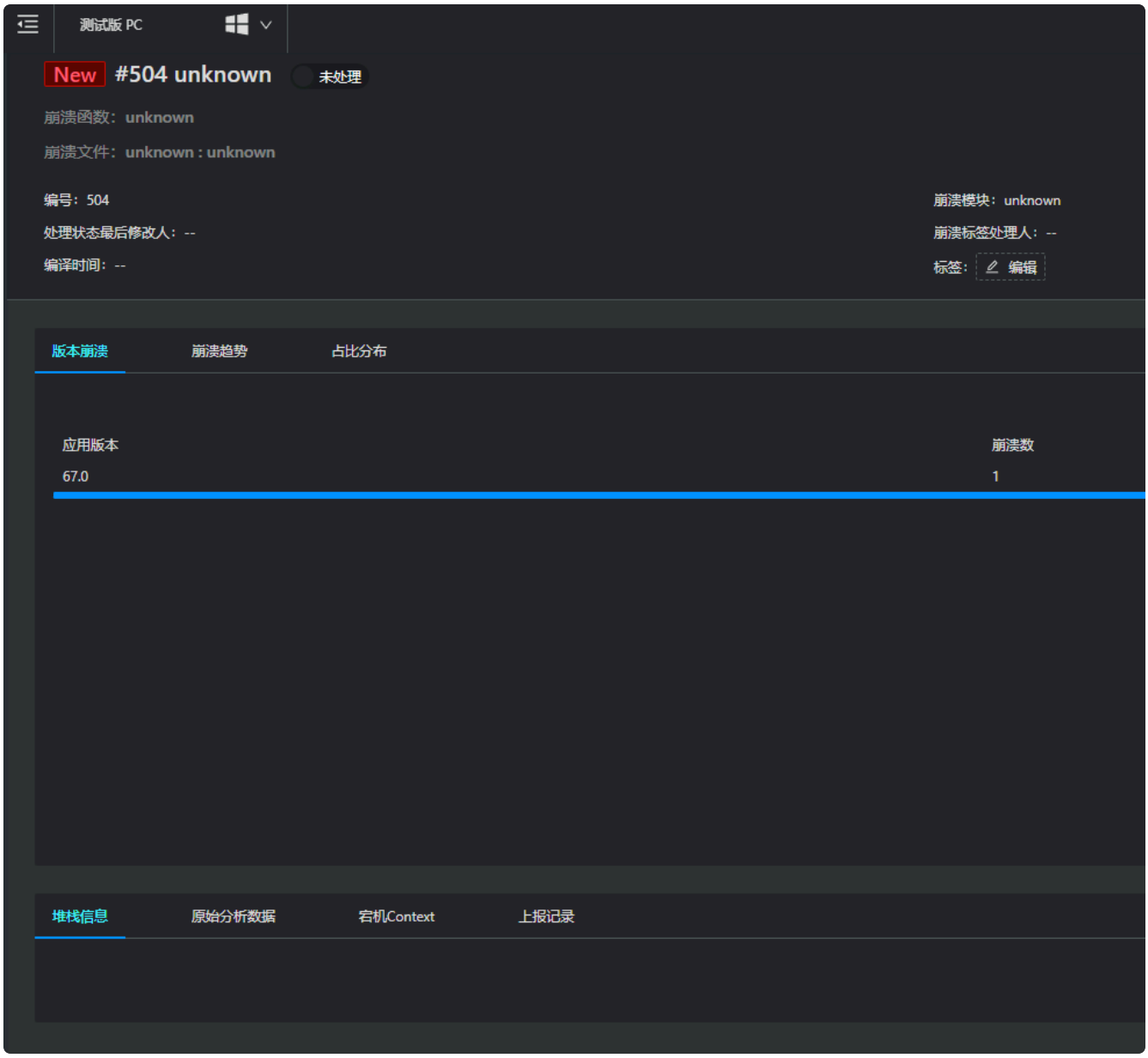
项目内接入了珠海质量中心的 CrashEye 插件，会在引擎层捕获 Crash 时将 Crash 相关信息上传到 CrashEye 平台，平台会自动解析 Crash 堆栈并打包 Log、MiniDump 等



CrashEye 平台会直接展示解析的 Crash 堆栈，通常一些常见比较简单的问题可以直接通过 Crash 调用堆栈就能定位到具体的问题

1.2 Log

CrashEye 偶尔会有 bug，Crash 堆栈解析失败，例如下面这种情况

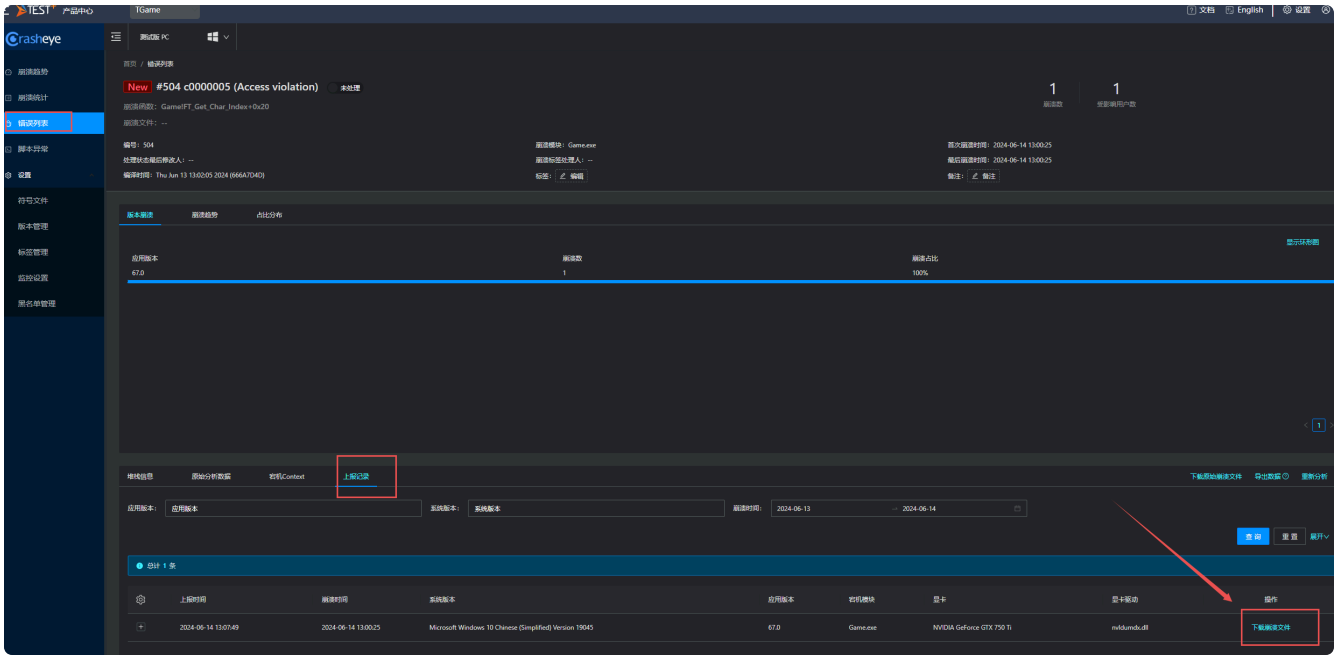


按照以前，遇到这种问题就无法定位了，因为以前接入 CrashEye 时的引擎修改会导致 Log 文件不会输出 Crash 堆栈信息

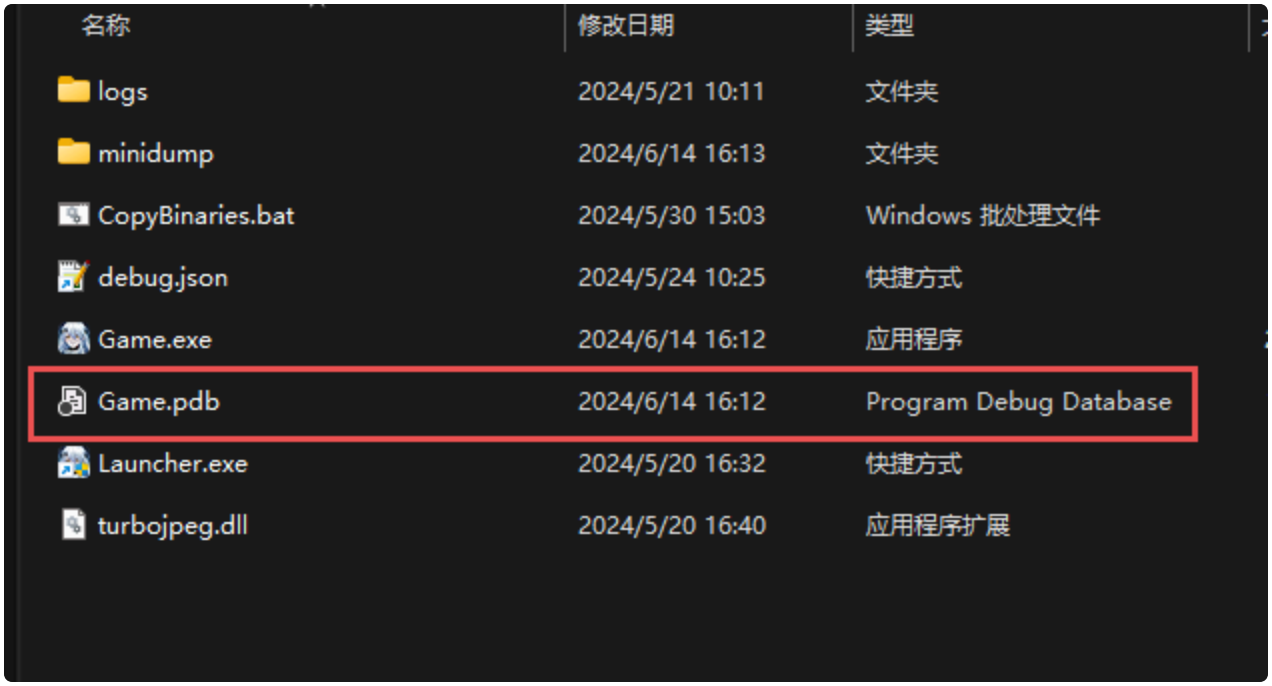
最近我在解决一些引擎优化问题时，同时将这个问题也修复了一下，Crash 堆栈会自动写入到 Log 文件中，就算 CrashEye 平台解析失败，也可以下载 Log 文件查看到

```
stack traceback:
[2024.06.13-12.31.29:818][  0]LogUnLua:
[2024.06.13-12.31.29:819][  0]LogWindows: Error: === Critical error: ===
[2024.06.13-12.31.29:819][  0]LogWindows: Error:
[2024.06.13-12.31.29:819][  0]LogWindows: Error: Fatal error!
[2024.06.13-12.31.29:819][  0]LogWindows: Error:
[2024.06.13-12.31.29:819][  0]LogWindows: Error: Unhandled Exception: EXCEPTION_ACCESS_VIOLATION reading address 0x0000000000000000
[2024.06.13-12.31.29:819][  0]LogWindows: Error:
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff6284f8b7d Game.exe!UGameApplication::Init() [F:\TGame\Proj\Source\Game\GameApplication.cpp:157]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff62d8bf627 Game.exe!UGameInstance::InitializeStandalone() [G:\Engine\dev\Engine\Source\Runtime\Engine\Private\GameInstance.cpp:158]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff62d7db308 Game.exe!UGameEngine::Init() [G:\Engine\dev\Engine\Source\Runtime\Engine\Private\GameEngine.cpp:1102]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff626335308 Game.exe!FEngineLoop::Init() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\LaunchEngineLoop.cpp:3996]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff62634ff5d Game.exe!GuardedMain() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\Launch.cpp:155]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff62635016a Game.exe!GuardedMainWrapper() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\Windows\LaunchWindows.cpp:137]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff626364220 Game.exe!WinMain() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\Windows\LaunchWindows.cpp:268]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ff6308e81fe Game.exe!__scrt_common_main_seh() [D:\a\_work\1\s\src\vctools\crt\vcstartup\src\startup\exe_common.inl:288]
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ffa11b653e0 KERNEL32.DLL!UnknownFunction []
[2024.06.13-12.31.29:819][  0]LogWindows: Error: [Callstack] 0x00007ffa12a2485b ntdll.dll!UnknownFunction []
[2024.06.13-12.31.29:819][  0]LogWindows: Error:
[2024.06.13-12.31.29:829][  0]LogDesigner: Warning: DesignerCore WorldEnd2 is successed!
[2024.06.13-12.31.29:831][  0]LogExit: Executing StaticShutdownAfterError
[2024.06.13-12.31.29:837][  0]LogWindows: FPlatformMisc::RequestExit(1)
[2024.06.13-12.31.29:837][  0]LogCore: Engine exit requested (reason: Win RequestExit)
[2024.06.13-12.31.29:839][  0]Log file closed, 06/13/24 20:31:29
```

Log 文件可以通过以下方式下载获取



PS：Log 文件中的 Crash 堆栈的输出是依赖引擎层的自动解析，因此需要保证对应平台的可执行文件目录下有对应版本的符号文件，例如 Windows 平台就是 PDB 文件



名称	修改日期	类型
logs	2024/5/21 10:11	文件夹
minidump	2024/6/14 16:13	文件夹
CopyBinaries.bat	2024/5/30 15:03	Windows 批处理文件
debug.json	2024/5/24 10:25	快捷方式
Game.exe	2024/6/14 16:12	应用程序
Game.pdb	2024/6/14 16:12	Program Debug Database
Launcher.exe	2024/5/20 16:32	快捷方式
turbojpeg.dll	2024/5/20 16:40	应用程序扩展

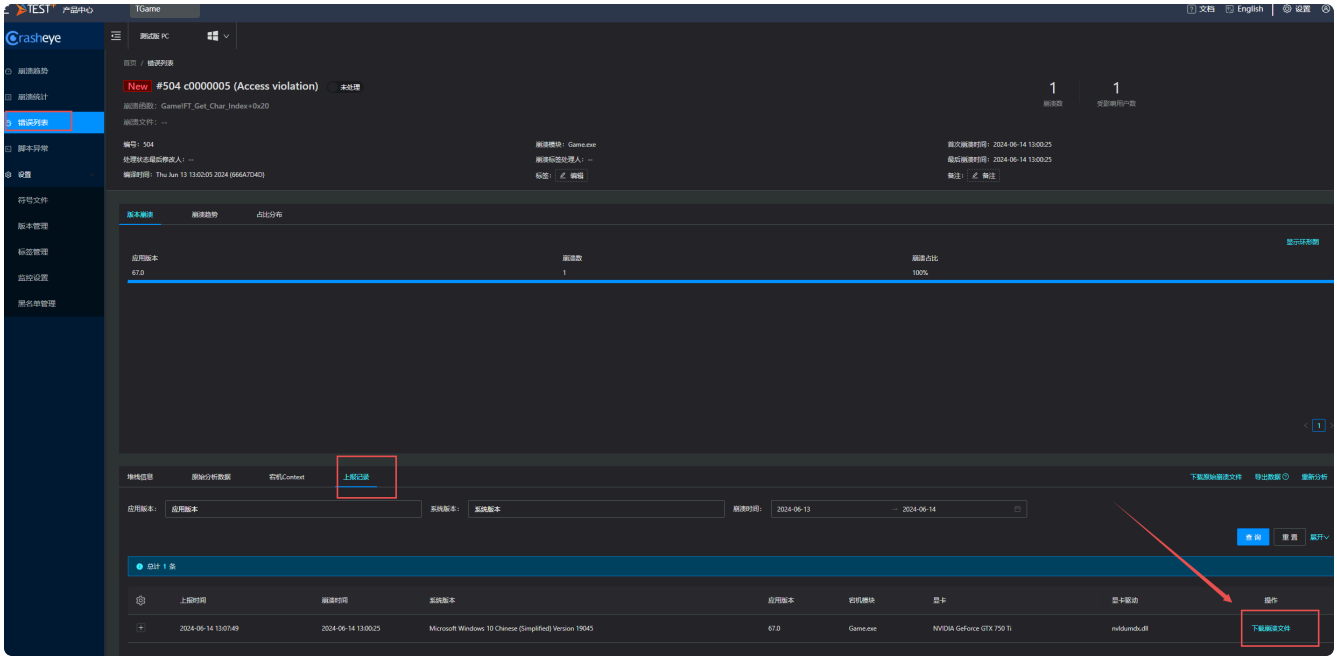
1.3 MiniDump

有一些比较棘手的 Crash，仅通过调用堆栈无法定位具体问题，这种情况需要用 MiniDump 恢复 Crash 现场进行 Debug

> 大多数Crash都能在Windows平台复现，因此这里以Windows平台为例，讲解如何通过MiniDump进行Debug

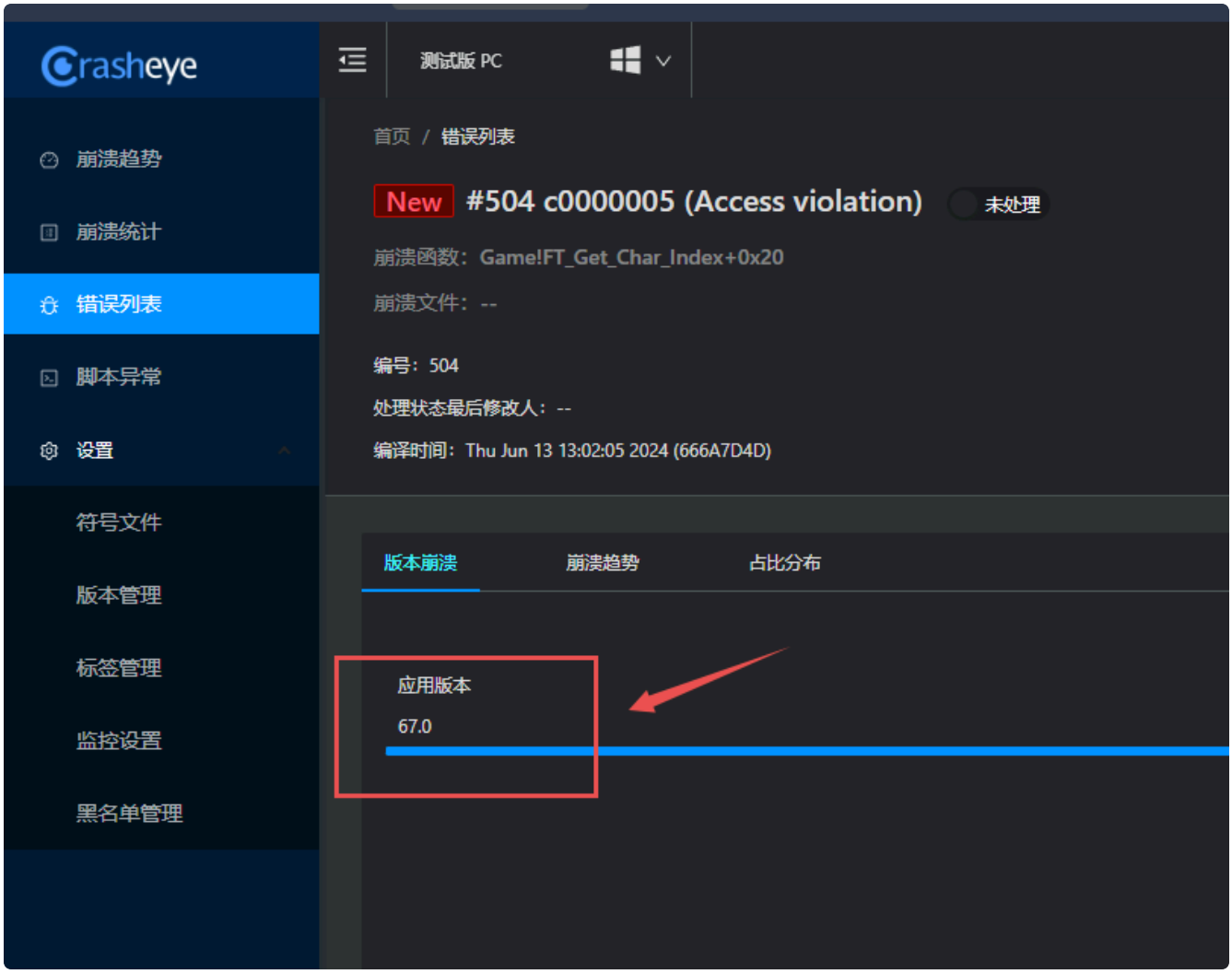
- **下载 MiniDump**

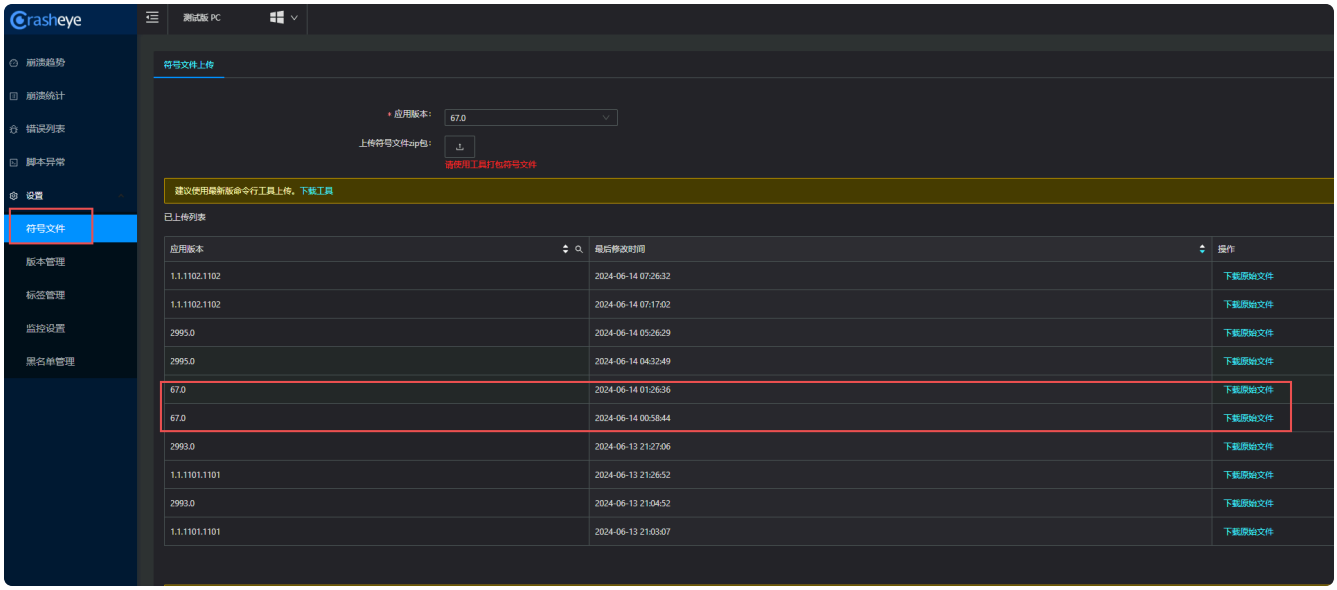
同下载 Log 文件一样

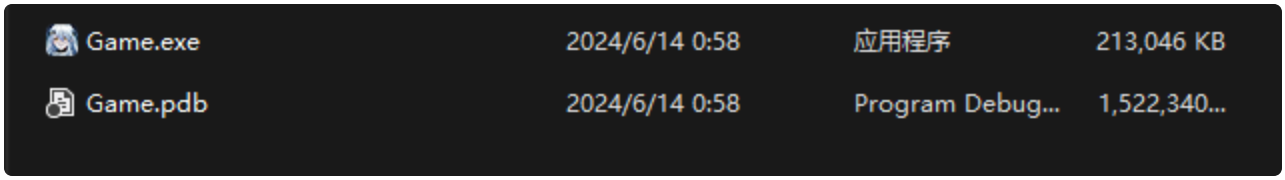


- **下载可执行文件和符号文件**

根据包体版本下载

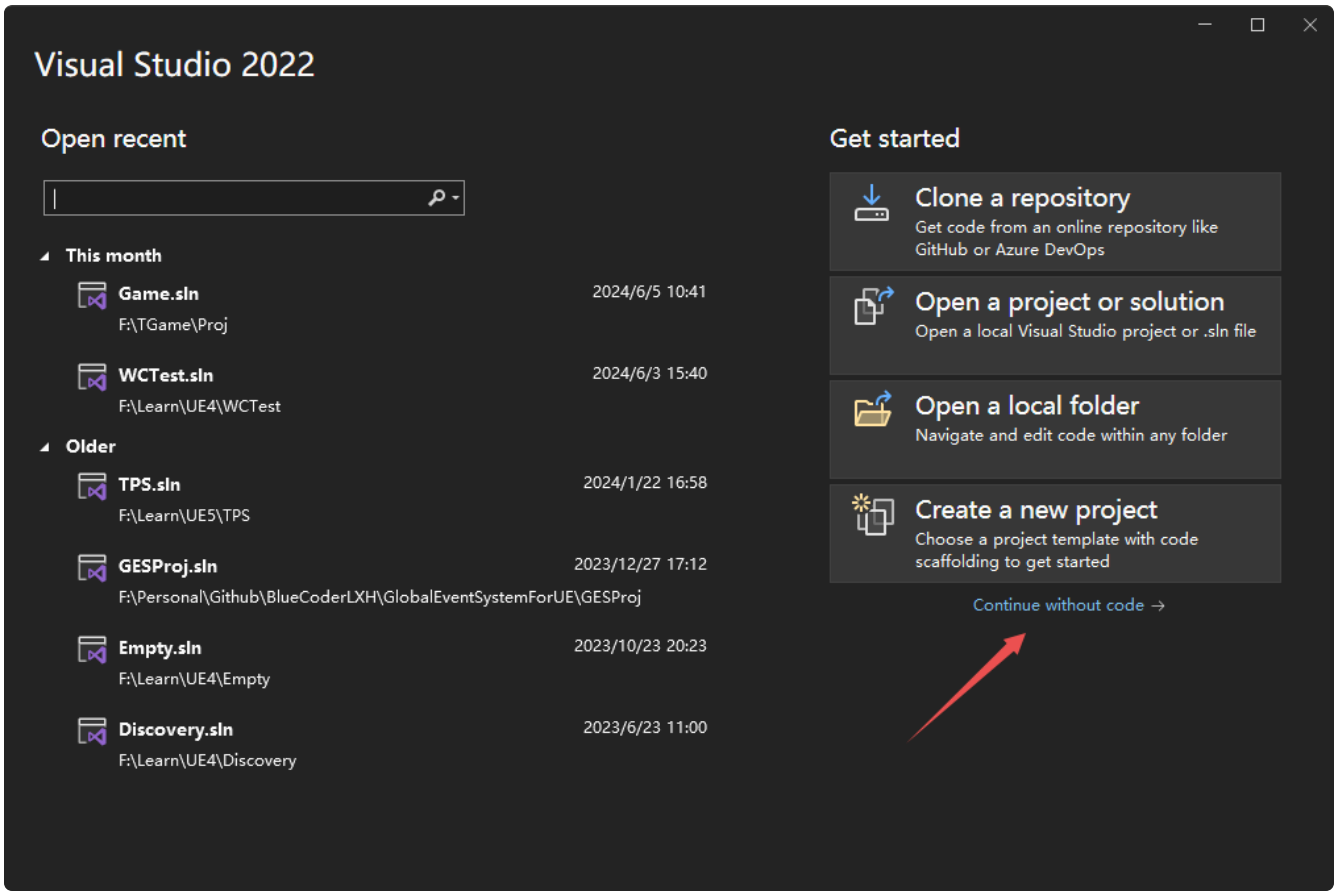

Crasheye
测试版 PC
崩溃趋势
崩溃统计
错误列表
脚本异常
设置
符号文件
版本管理
标签管理
监控设置
黑名单管理
首页 / 错误列表
New #504 c0000005 (Access violation)
未处理
崩溃函数：Game!FT_Get_Char_Index+0x20
崩溃文件：--
编号：504
处理状态最后修改人：--
编译时间：Thu Jun 13 13:02:05 2024 (666A7D4D)
版本崩溃
崩溃趋势
占比分布
应用版本
67.0


Crasheye
测试版 PC
崩溃趋势
崩溃统计
错误列表
脚本异常
设置
符号文件
版本管理
标签管理
监控设置
黑名单管理
符号文件上传
* 应用版本：
67.0
上传符号文件zip包：
请使用工具打包符号文件
建议使用最新版命令行工具上传。下载工具
已上传列表
应用版本
最后修改时间
操作
1.1.1102.1102
2024-06-14 07:26:32
下载原始文件
1.1.1102.1102
2024-06-14 07:17:02
下载原始文件
2995.0
2024-06-14 05:26:29
下载原始文件
2995.0
2024-06-14 04:32:49
下载原始文件
67.0
2024-06-14 01:26:36
下载原始文件
67.0
2024-06-14 00:58:44
下载原始文件
2993.0
2024-06-13 21:27:06
下载原始文件
1.1.1101.1101
2024-06-13 21:26:52
下载原始文件
2993.0
2024-06-13 21:04:52
下载原始文件
1.1.1101.1101
2024-06-13 21:03:07
下载原始文件

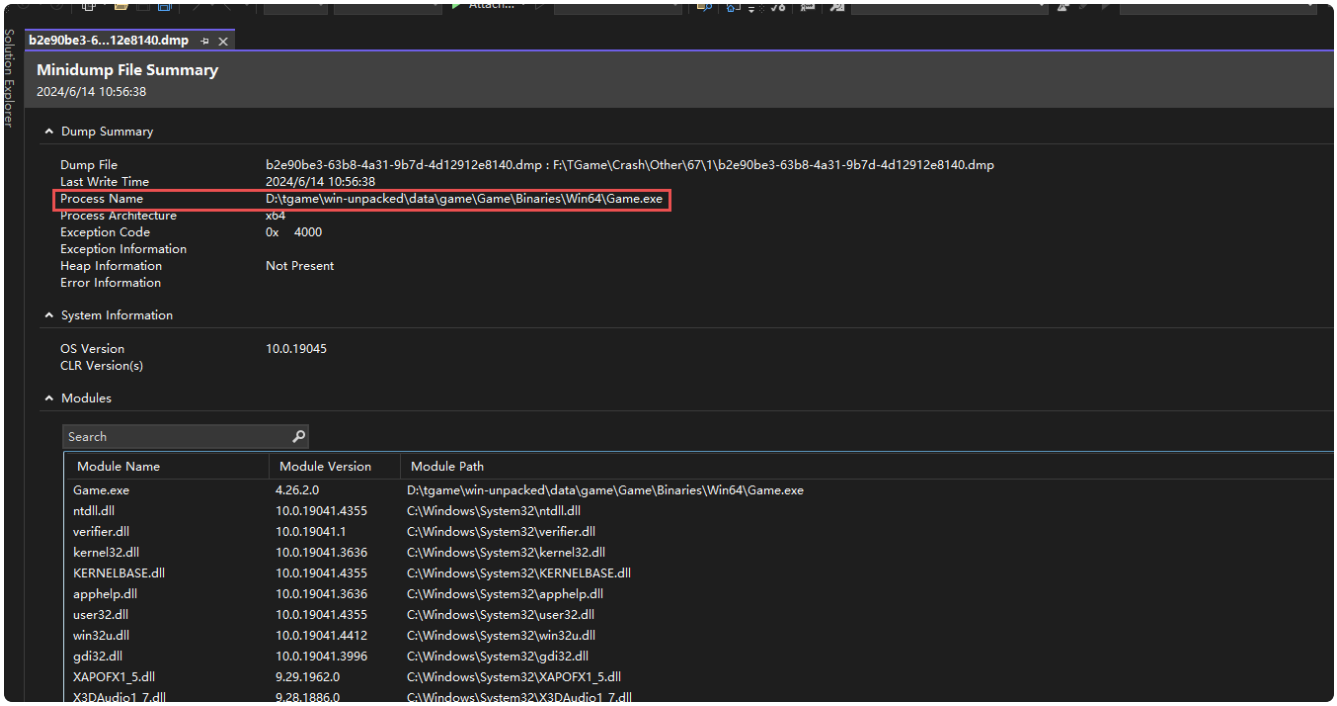


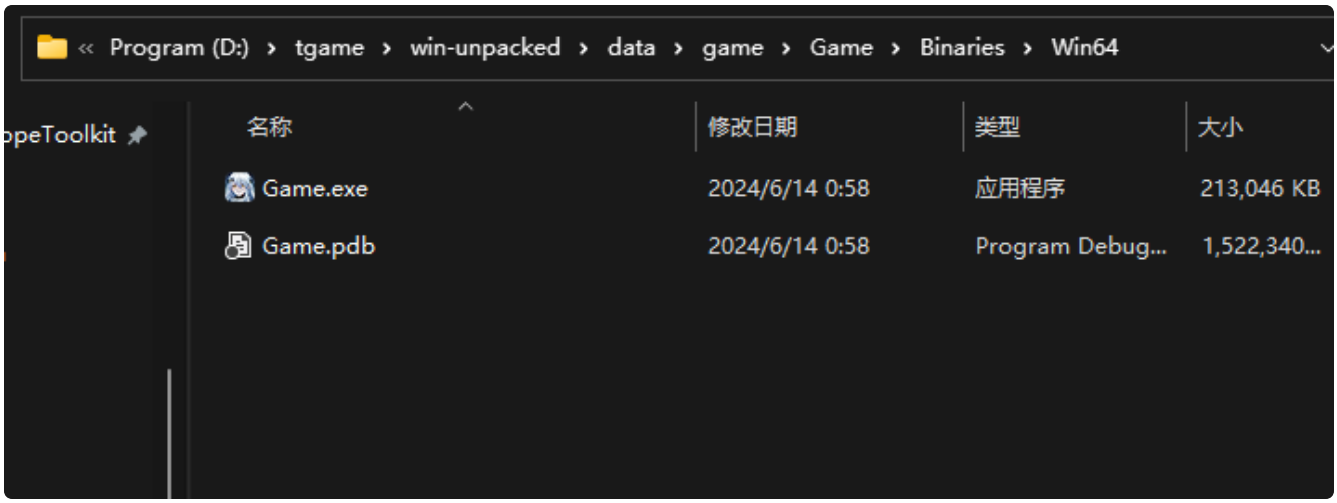
- **VS 打包 Dump 文件**

直接打开 VS

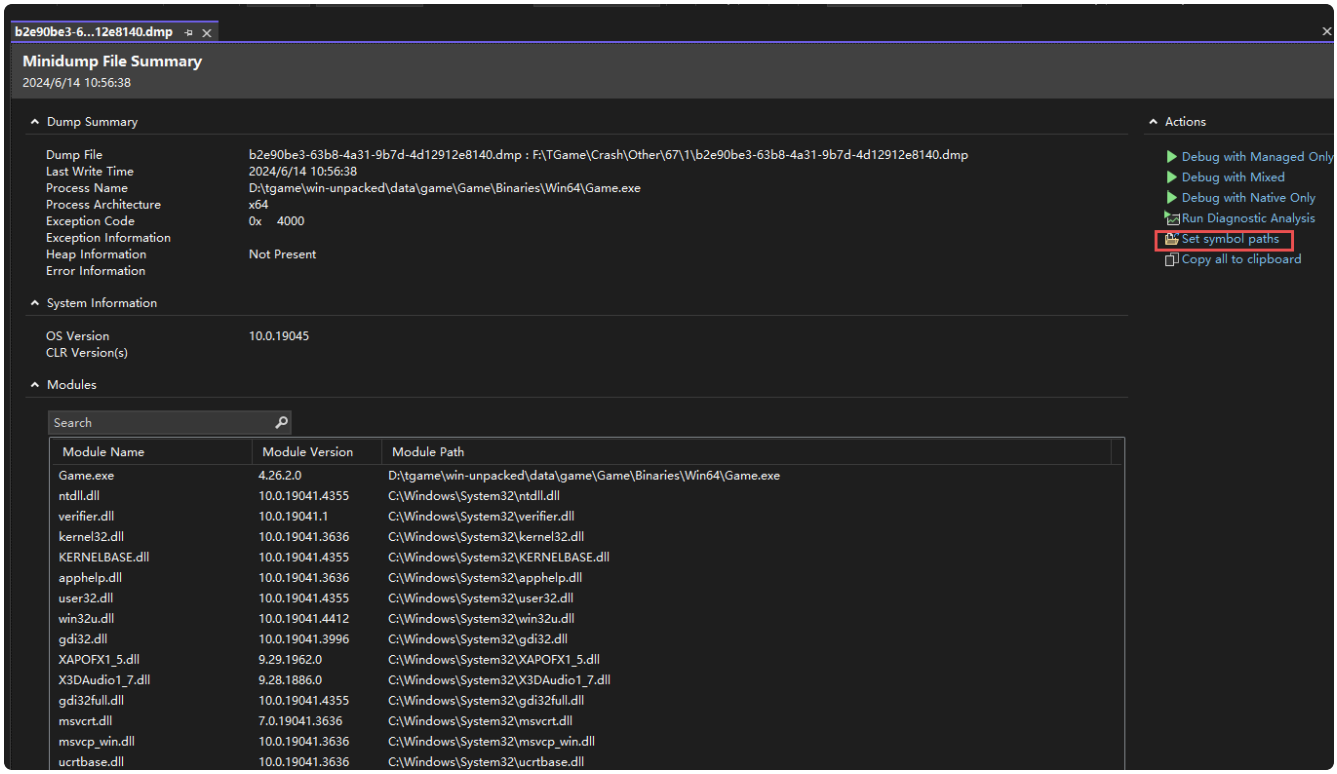


将 Dump 文件拖拽 VS，并将下载的可执行文件(Exe)和符号文件(PDB)拷贝到显示的进程名目录（也就是可执行文件所在目录）

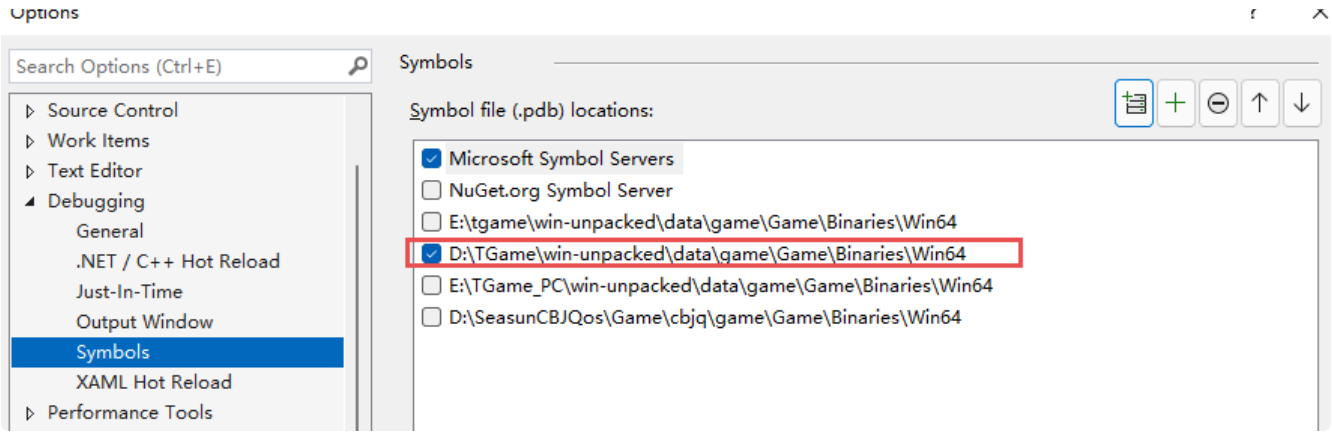

b2e90be3-6...12e8140.dmp
Minidump File Summary
2024/6/14 10:56:38
Dump Summary
Dump File b2e90be3-63b8-4a31-9b7d-4d12912e8140.dmp : F:\TGame\Crash\Other\67\1\b2e90be3-63b8-4a31-9b7d-4d12912e8140.dmp
Last Write Time 2024/6/14 10:56:38
Process Name D:\tgame\win-unpacked\data\game\Game\Binaries\Win64\Game.exe
Process Architecture x64
Exception Code 0x 4000
Exception Information
Heap Information Not Present
Error Information
System Information
OS Version 10.0.19045
CLR Version(s)
Modules
Search
Module Name Module Version Module Path
Game.exe 4.26.2.0 D:\tgame\win-unpacked\data\game\Game\Binaries\Win64\Game.exe
ntdll.dll 10.0.19041.4355 C:\Windows\System32\ntdll.dll
verifier.dll 10.0.19041.1 C:\Windows\System32\verifier.dll
kernel32.dll 10.0.19041.3636 C:\Windows\System32\kernel32.dll
KERNELBASE.dll 10.0.19041.4355 C:\Windows\System32\KERNELBASE.dll
apphelp.dll 10.0.19041.3636 C:\Windows\System32\apphelp.dll
user32.dll 10.0.19041.4355 C:\Windows\System32\user32.dll
win32u.dll 10.0.19041.4412 C:\Windows\System32\win32u.dll
gdi32.dll 10.0.19041.3996 C:\Windows\System32\gdi32.dll
XAPOFX1_5.dll 9.29.1962.0 C:\Windows\System32\XAPOFX1_5.dll


Program (D:) › tgame › win-unpacked › data › game › Game › Binaries › Win64
名称 修改日期 类型 大小
Game.exe 2024/6/14 0:58 应用程序 213,046 KB
Game.pdb 2024/6/14 0:58 Program Debug... 1,522,340...

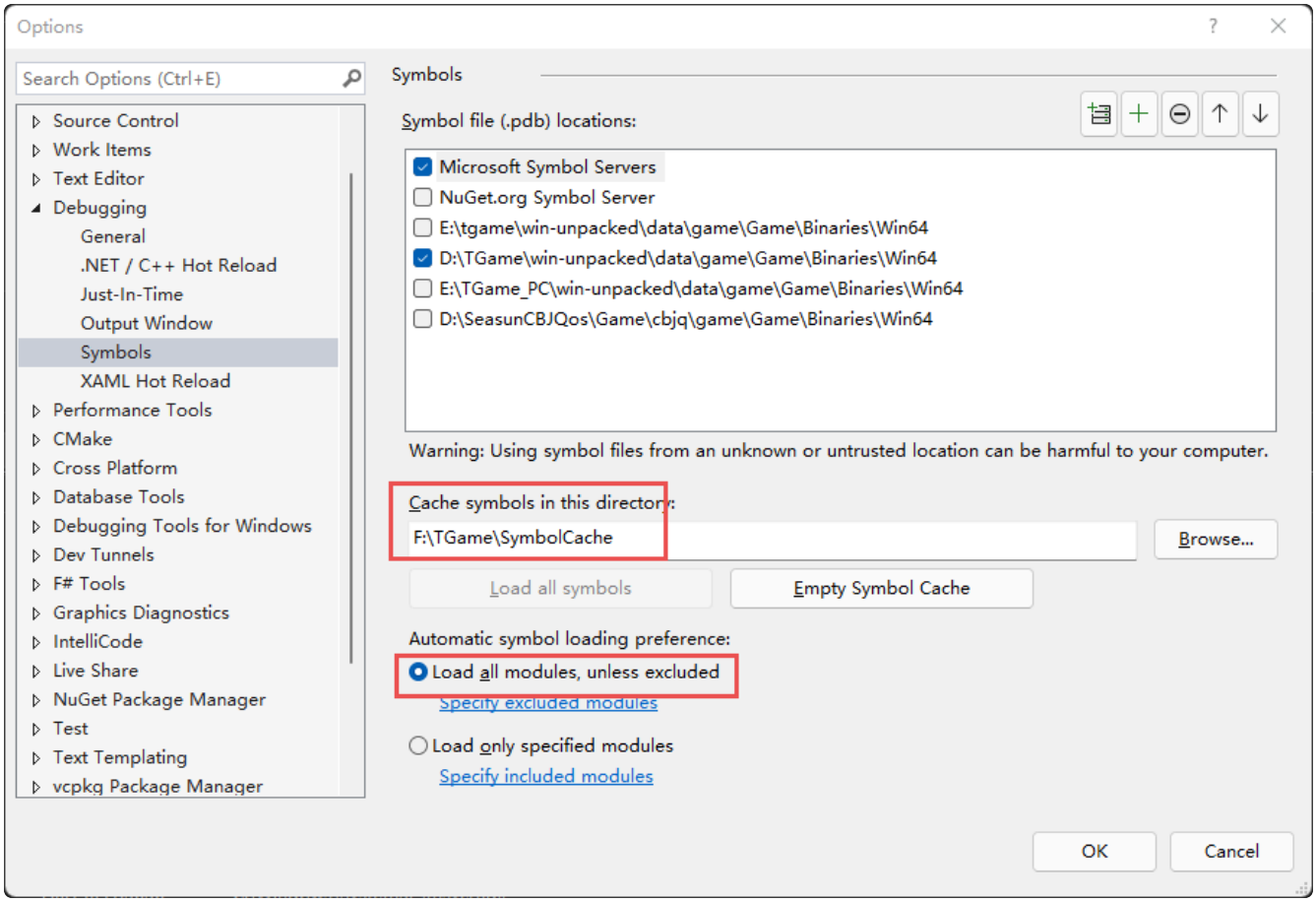
- 设置符号文件所在目录



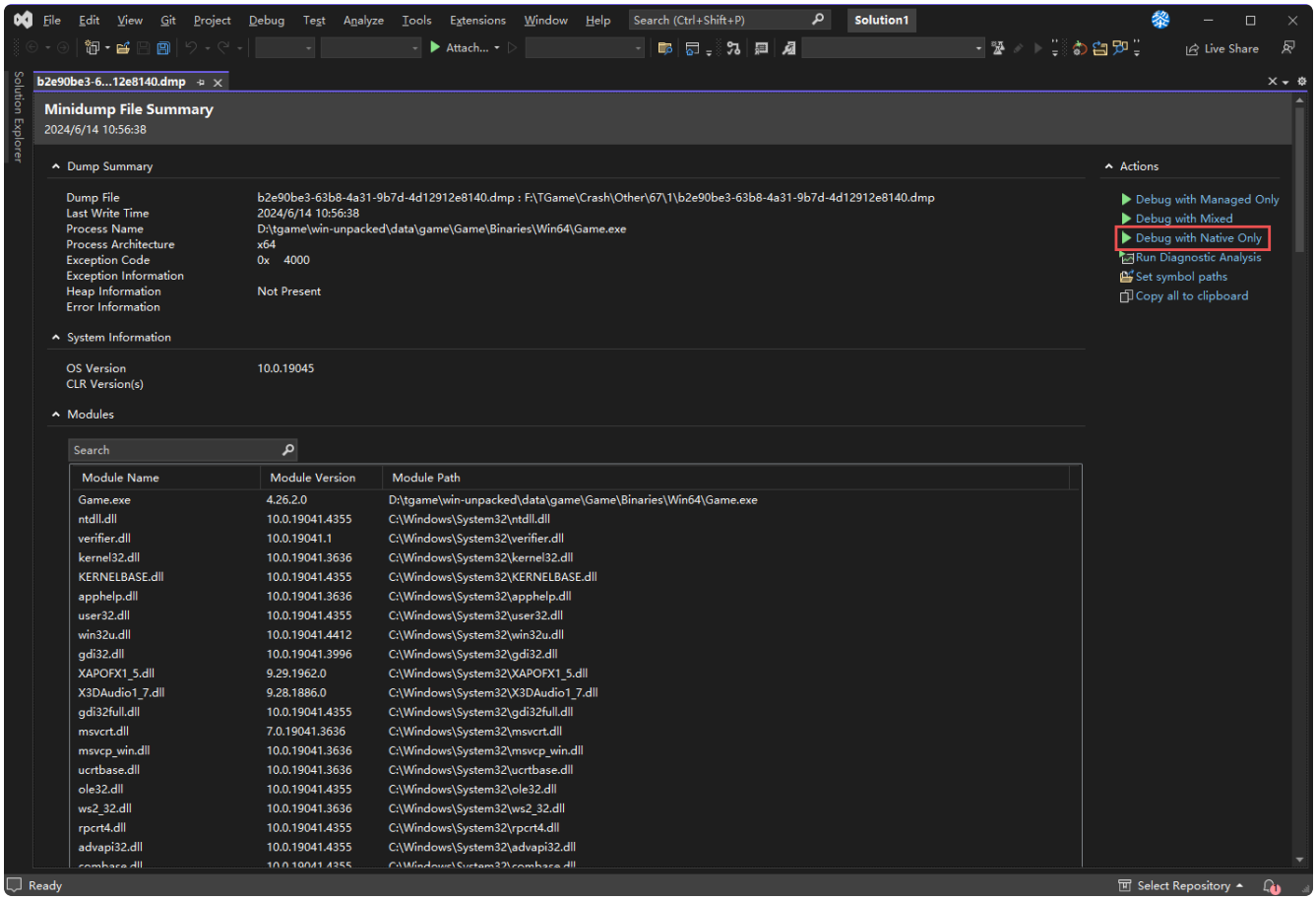
项目的 PDB 就设为上一步拷贝的目录



windows 下，还有一些系统的 dll 和 pdb 需要从 microsoft pdb cache server 端下载，因此这里需要设置一下缓存目录，避免每次都从远端下载

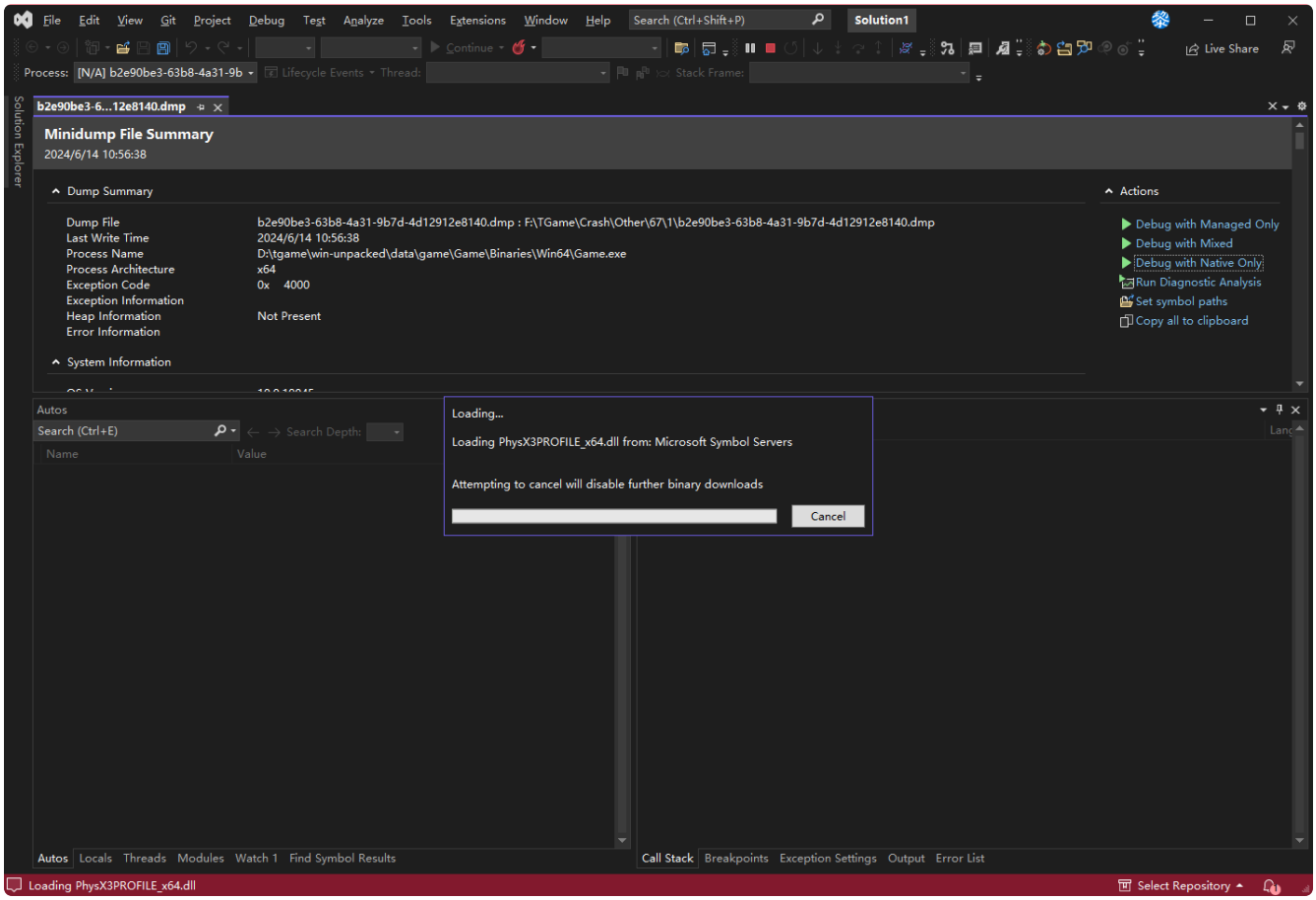

Options
Search Options (Ctrl+E)
Source Control
Work Items
Text Editor
Debugging
General
.NET / C++ Hot Reload
Just-In-Time
Output Window
Symbols
XAML Hot Reload
Performance Tools
CMake
Cross Platform
Database Tools
Debugging Tools for Windows
Dev Tunnels
F# Tools
Graphics Diagnostics
IntelliCode
Live Share
NuGet Package Manager
Test
Text Templating
vcpkg Package Manager
Symbols
Symbol file (.pdb) locations:
Microsoft Symbol Servers
NuGet.org Symbol Server
E:\tgame\win-unpacked\data\game\Game\Binaries\Win64
D:\TGame\win-unpacked\data\game\Game\Binaries\Win64
E:\TGame_PC\win-unpacked\data\game\Game\Binaries\Win64
D:\SeasunCBJQos\Game\cbjq\game\Game\Binaries\Win64
Warning: Using symbol files from an unknown or untrusted location can be harmful to your computer.
Cache symbols in this directory:
F:\TGame\SymbolCache
Browse...
Load all symbols
Empty Symbol Cache
Automatic symbol loading preference:
Load all modules, unless excluded
Specify excluded modules
Load only specified modules
Specify included modules
OK
Cancel

- 开始 Debug



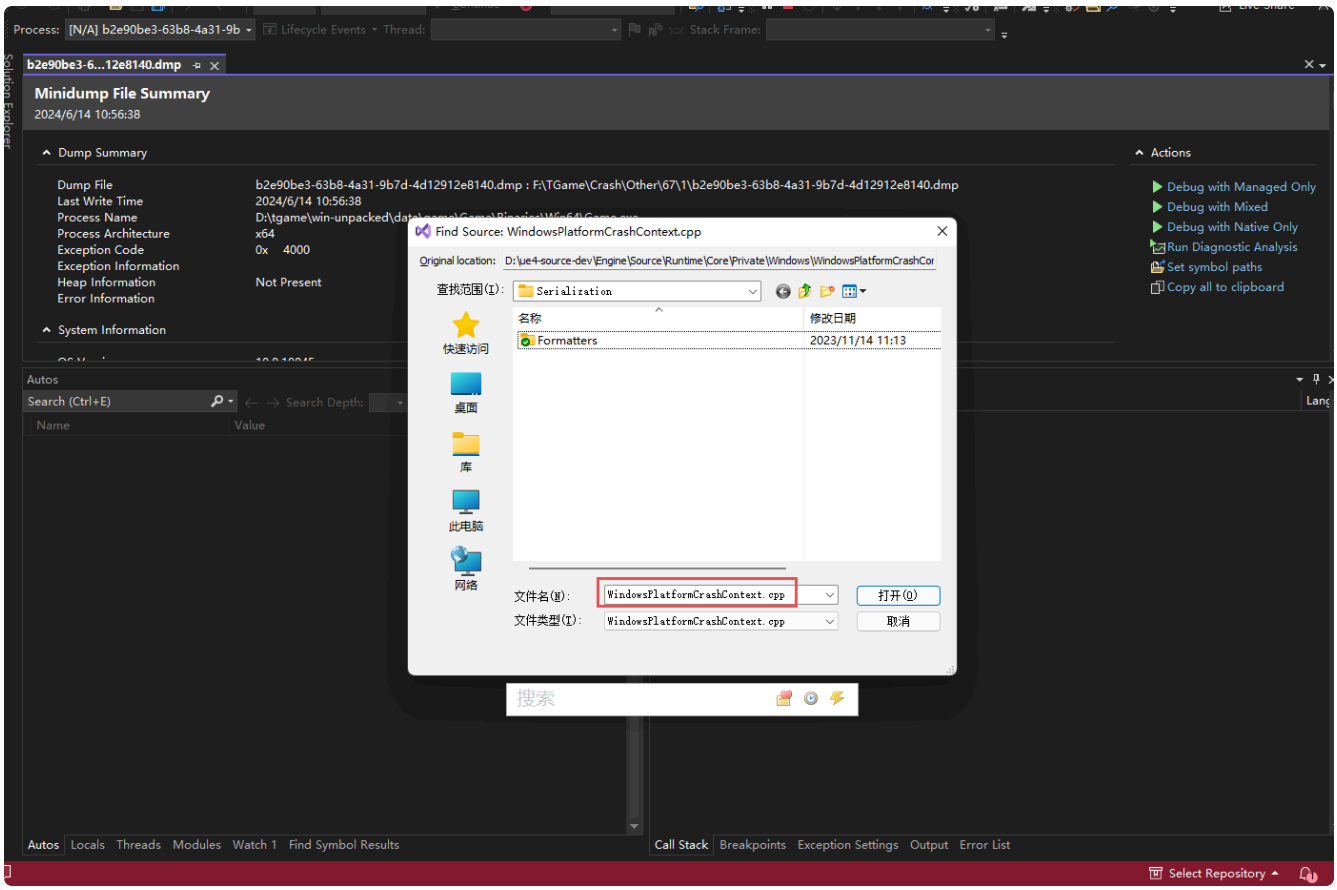
这里会先从远端下载所需的系统 dll 和 pdb

> 第一次下载可能需要很长一段时间，后续有需要可以在内网建立一个cache server

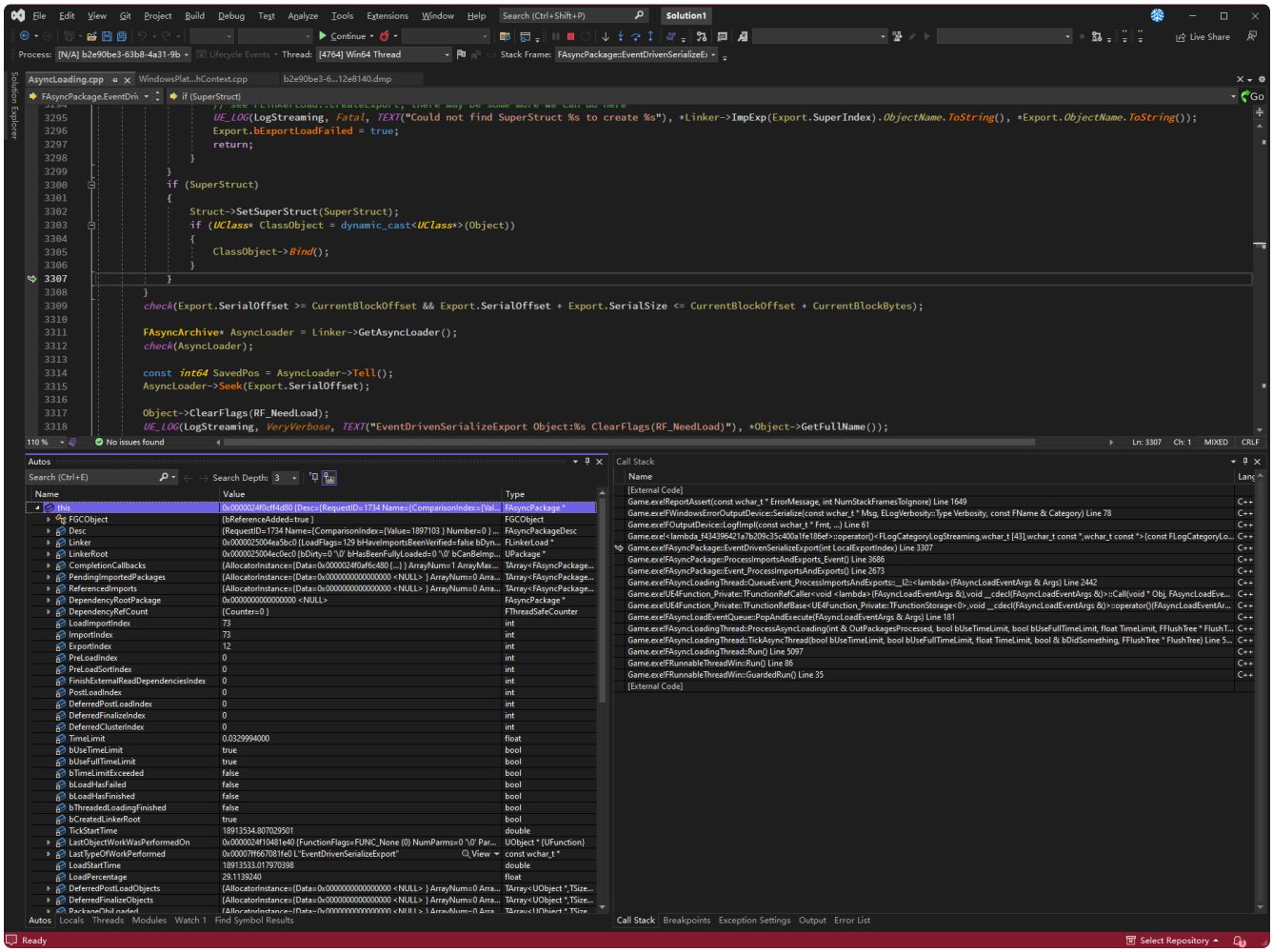


- 设置代码源文件目录

设置为本地引擎和项目对应代码所在目录即可



- 以上都正确操作完后，即可恢复 Crash 现场，查看调用堆栈的详细信息，包括 UObject、Class 等等

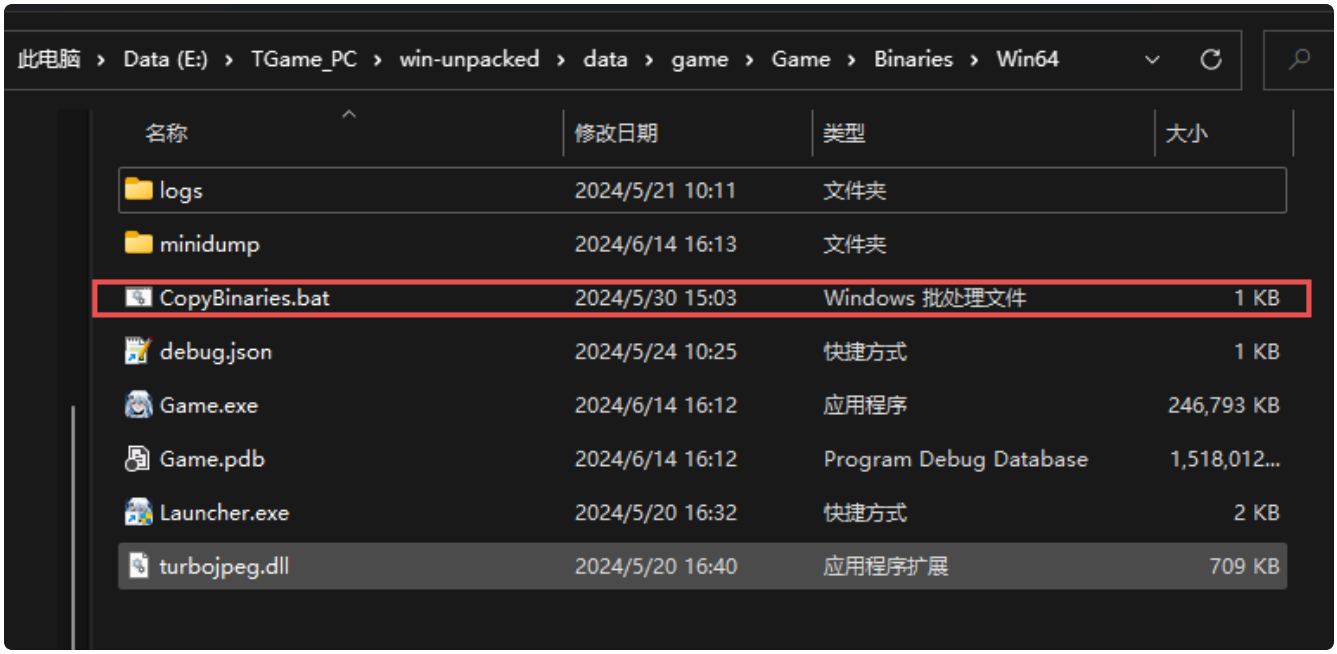


1.4 本地修改代码进行 Debug

通过前面的手段基本能定位到绝大多数 Crash 问题的原因所在，但依然还存在一些很难定位的疑难杂症，我的终极大法就是直接本地修改代码进行 Debug，这种方式的好处是一方面可以看到更加完整的堆栈信息，另一方面是可以修改代码立马进行验证和 Debug

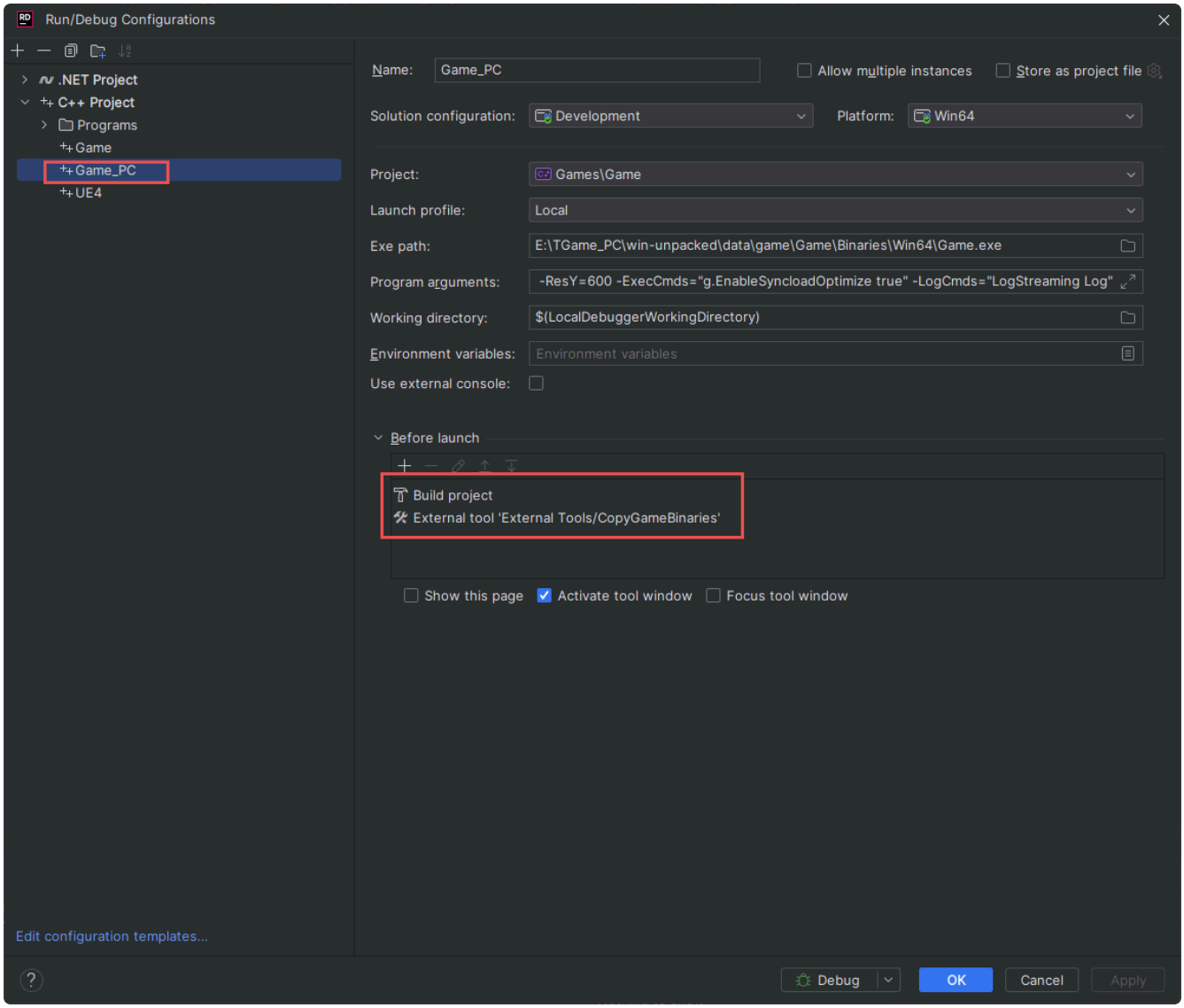
具体步骤如下：

- **首先下载内网打的 Windows 包**
- **写一个简单的脚本自动拷贝 EXE 和 PDB**



- **IDE 创建专属的 Debug 配置**

> 我本地使用的Rider，以Rider为例，VS类似



并建立两个启动任务：

① Build project：编译工程，这一步会生成 EXE 和 PDB

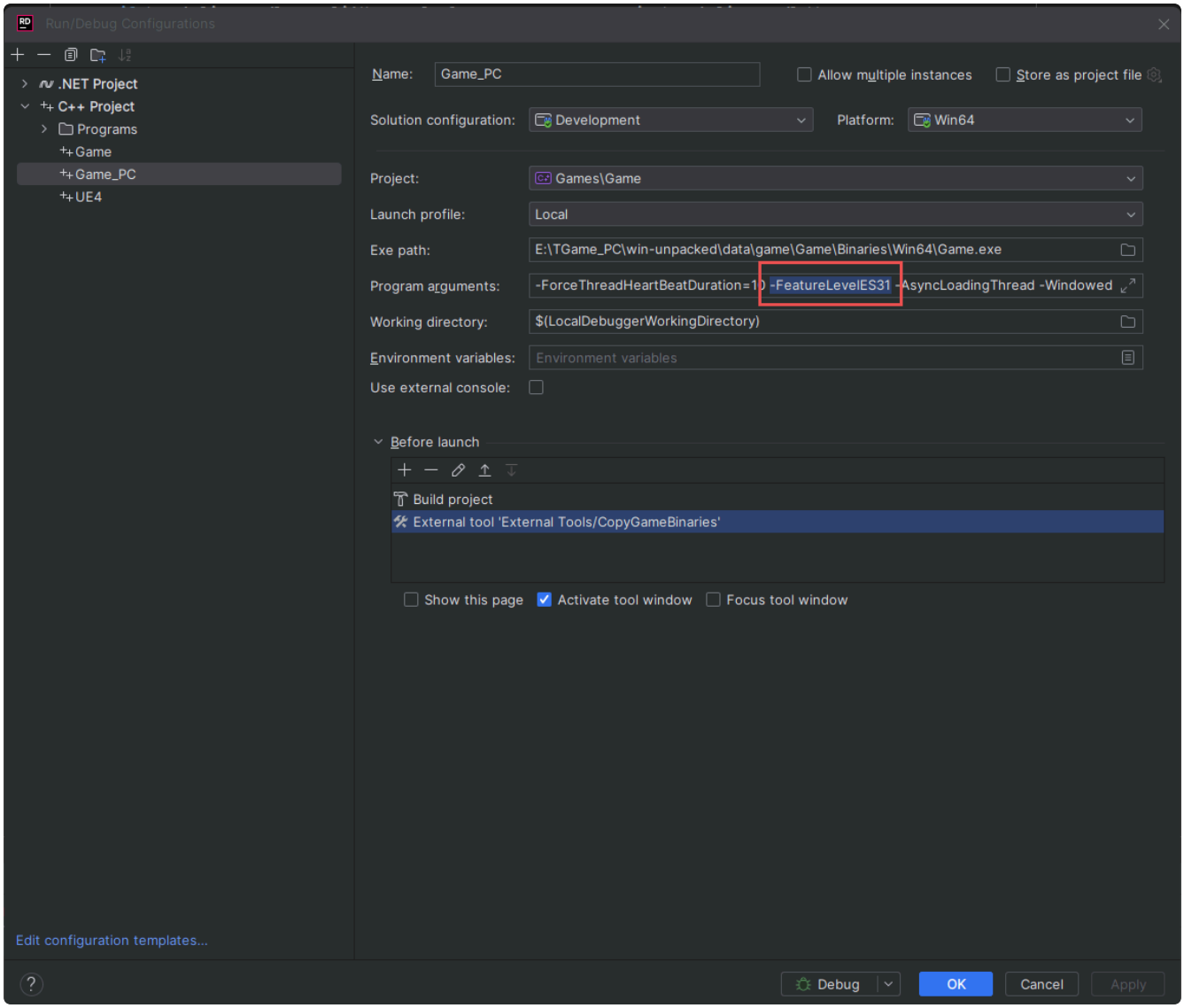
② External tool 'External Tools/CopyGameBinaries'： 自动将生成的 EXE 和 PDB 拷贝到下载的内网包所在目录

这样本地修改完代码后，会自动将编译好的 EXE 和 PDB 拷贝到内网包所在目录，也就可以直接用 IDE 进行 Debug

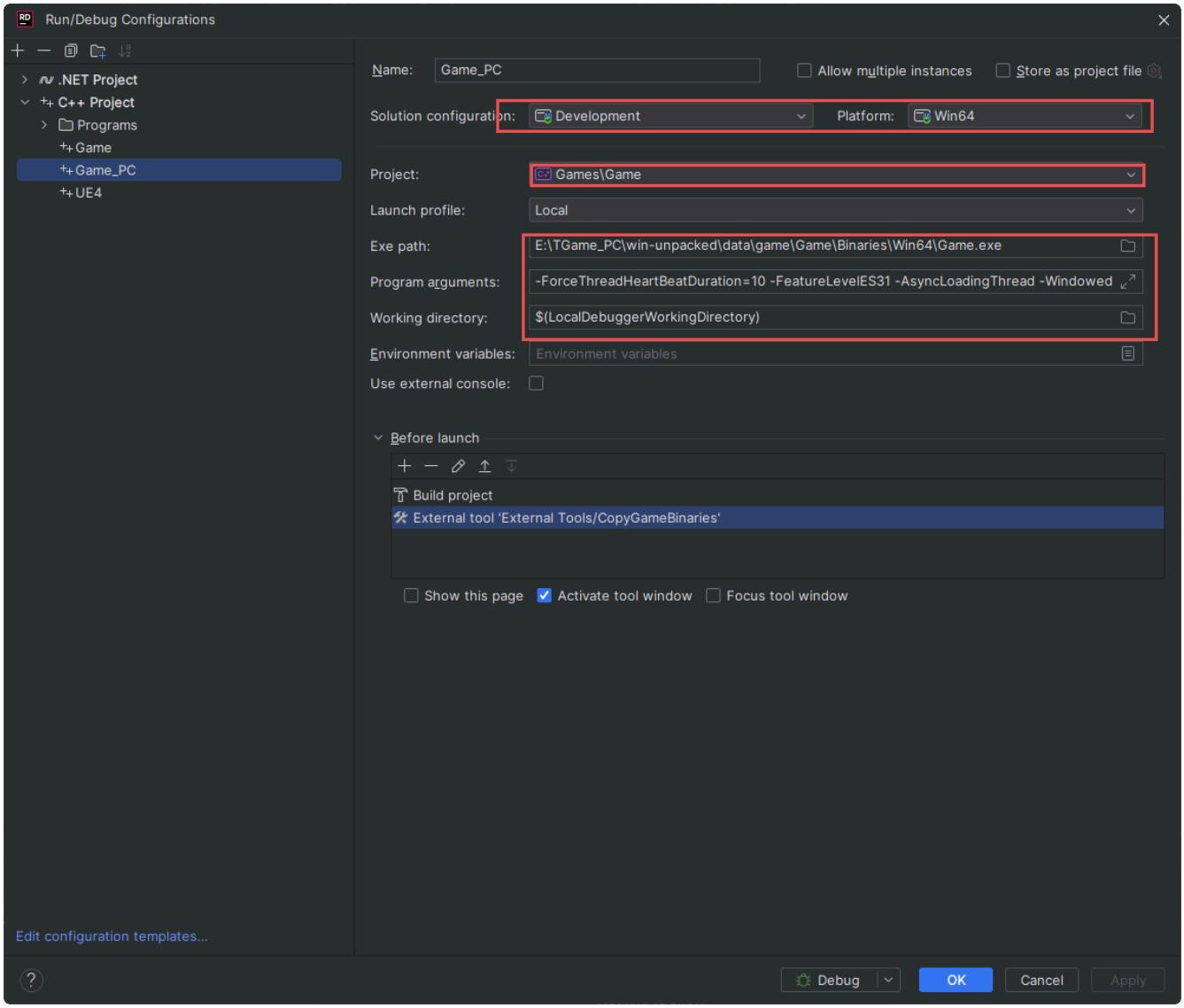
- **不要从 Launcher 启动，而是直接启动 Game**

这个需要增加启动参数

> -FeatureLevelES31



- 其他配置按照本地情况来设置即可



如果遇到一些变量被优化无法查看，可以将对应 Module 的代码优化关闭：

```
C# CoreUObject.Build.cs ×
3   using UnrealBuildTool;
4
    ? usages  ? inheritors  Tim Sweeney +3 *  ? extension methods  ? exposing APIs
5   public class CoreUObject : ModuleRules
6   {
        ? usages  ? overrides  Tim Sweeney +3 *  ? extension methods  ? exposing APIs
7       public CoreUObject(ReadOnlyTargetRules Target) : base(Target)
8       {
9           OptimizeCode = CodeOptimization.Never;
10
11          PrivatePCHHeaderFile = "Private/CoreUObjectPrivatePCH.h";
12
13          SharedPCHHeaderFile = "Public/CoreUObjectSharedPCH.h";
14
15          PrivateIncludePaths.Add( item: "Runtime/CoreUObject/Private");
16
17          PrivateIncludePathModuleNames.AddRange(
18                  collection: new string[]
19                  {
20                      "TargetPlatform",
21                  }
22              );
23
24          PublicDependencyModuleNames.Add( item: "Core");
25          PublicDependencyModuleNames.Add( item: "TraceLog");
26
27          PrivateDependencyModuleNames.Add( item: "Projects");
28          PrivateDependencyModuleNames.Add( item: "Json");
29
30      }
31
32  }
33
```

二、卡死 / 死循环

最近整合了一下引擎层 ThreadHeartBeat 功能，如果遇到卡死或死循环的情况，可以用过以下命令直接打开：

> -ForceThreadHeartBeatDuration=10

引擎底层会创建一个守护进程自动检测线程卡死，当出现卡死 10s 后，会在 log 中输出堆栈信息“LogCore: Error: Hang detected on xxxThread”，方便快速定位到卡死或死循环逻辑

类似这种：

```
LogCore: Error: Hang detected on GameThread (thread hasn't sent a heartbeat for 10.00 seconds):
LogCore: Error:   0x00007ff615607147 Game.exe!FEngineLoop::Init() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\LaunchEngineLoop
LogCore: Error:   0x00007ff61562125d Game.exe!GuardedMain() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\Launch.cpp:155]
LogCore: Error:   0x00007ff615621f9a Game.exe!WinMain() [G:\Engine\dev\Engine\Source\Runtime\Launch\Private\Windows\LaunchWindows.cpp:
LogCore: Error:   0x00007ff61fad1bee Game.exe!__scrt_common_main_seh() [D:\a\_work\1\s\src\vctools\crt\vcstartup\src\startup\exe_commo
LogCore: Error:   0x00007ffb688553e0 KERNEL32.DLL!UnknownFunction []
LogCore: Error:   0x00007ffb69d8485b ntdll.dll!UnknownFunction []
```